ENGEN582-25X
Honours Research and Development Project



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Sustainability as a Software Engineering Problem

Aryan Thanki

Judy Bowen

Table of Contents

1. Introduction 2
2. Sustainability in Computing 3
2.1 Traditional Focus on Hardware 3
2.2 The Growing Environmental Burden of Software 3
2.3 Energy Consumption Trends in Modern ICT 3
2.4 System Effects and Rebound Phenomena 4
2.5 Need for a Holistic Sustainability Approach 5
3. Foundations of Green Software Engineering 6
3.1 Defining Green and Sustainable Software Engineering 6
3.2 Green by IT versus Green In IT 6
3.3 Major Frameworks and Models 6
3.4 Challenges in Defining Metrics 8
4. Software Development Life Cycle 9
4.1 Requirements and Design Stages 9
4.2 Development and Testing Stages 10
4.3 Deployment, Maintenance and Disposal Stages 10
4.4 Metrics for Sustainable Software 11
5. Software Abandonment and Large-Scale Failures 12
5.1 Causes of Abandonment 12
5.2 FBI Virtual Case File 12
5.3 UK e-Borders 12
5.4 Taurus 12
5.5 Reason for project failure 13
6. Green Software Adoption in Industry Challenges 14
6.1 Economic and Organisational Constraints 14
6.2 Lack of Awareness and Tooling 14
6.3 Cultural and Technical Misconceptions 15
6.4 Trade-offs and Complexity in Real-World Projects 15
7. Summary 16
References 17

1. Introduction

Information and Communication Technology (ICT) is a broad category that involves many subsets of industry. With the growing demand for large, complex systems comes an equally increasing carbon footprint because of modern software development. While significant efforts have been made to control sustainability at the hardware level over the years, such as developing energy-efficient processors or promoting e-waste management (Naumann et al., 2011), the environmental footprint of software systems has grown significantly and is a growing contributor to global carbon emissions (Becker et al., 2015; Eeckhout, 2025).

Modern software applications, cloud computing infrastructure, and artificial intelligence are just some examples of the immense workload that the energy industry faces to satisfy computational resources. These demands aren't limited to development alone but also persist through deployment, maintenance, and everyday operation (Vartziotis et al., 2024). Moreover, software-induced obsolescence, where evolving software can lead to hardware becoming prematurely unusable, adds unnecessary quantities of e-waste (Hilty et al., 2006). Studies have shown that even as individual device efficiency improves, total energy efficiency does not rise, with consumption becoming greater due to a rebound effect and escalating usage patterns (Hilty et al., 2006; Eeckhout, 2025).

Recognising this, the field of Green and Sustainable Software Engineering was born, with the intent of developing software that is intrinsically sustainable ("Green in IT") but also for designing software that enables sustainability elsewhere (Dick & Naumann, 2010; Naumann et al., 2011). Foundation models have been created, such as the GREENSOFT model (Naumann et al., 2011), the Green Software Development model (Shenoy & Eeratta, 2011), or the Karlskrona Manifesto (Becker et al., 2015), which provide a framework for sustainability embedded throughout the software lifecycle, from requirements gathering all the way to the disposal phase. In conjunction, analysis tools like the Green Tracker (Amsel & Tomlinson, 2010) and Greenalytics (Zapico et al., 2010) for observing energy statistics have been designed to visualise and measure the environmental impacts of software.

Despite these advancements, significant gaps exist between research targets and real-world implementation. Adoption of green software engineering practices in industry remains limited, with barriers such as lack of economic incentives, limited developer awareness, or lack of integration into Agile working environments (Kern, 2017; Deng & Ji, 2015). Furthermore, large-scale project failures such as the FBI Virtual Case File or the UK e-Borders systems (Marchewka, 2014; Alami, 2016a) showcase the economic burdens of unstructured, unsustainable software practices.

This literature review will aim to critically examine the intersection between sustainability and software engineering. It will explore foundational concepts, models, and measurement tools; assess practices around the Software Development Life Cycle (SDLC); and analyse key barriers to adoption in the industry. Investigation of case studies involving large-scale software abandonment will be shown to highlight the real-world consequences of neglecting software development practices.

The key research questions this review will answer are

1. How has the understanding of sustainability developed in software, and what frameworks best capture its evolution?
2. At which stages of the software development life cycle is sustainability most successfully embedded?
3. What are the primary barriers to employing sustainable software development practices, and how can they be overcome?

By synthesising insights from foundational and emerging literature, this review seeks to outline the path forward by increasing knowledge surrounding sustainability as a software engineering problem.

2. Sustainability in Computing

2.1 Traditional Focus on Hardware

Early debates surrounding sustainability in the Information and Communication Technology (ICT) space had a skewed, hardware-centric focus. Environmental concerns were mainly associated with the material aspects of computing, such as the energy consumption of servers, e-waste generated by obsolete devices, or the resource-intensive aspects involved in electronics manufacturing (Naumann et al., 2011). Efforts to address these issues led to advancements in hardware design like energy-efficient processors, improved power management systems, and hardware recycling programmes (Bekaroo et al., 2013). Energy star certifications and other forms of eco-labelling initiatives for devices became common practice, encouraging manufacturers to pursue incremental gains in device manufacturing for energy efficiency (Mayr et al, 2016)

However, this hardware-focused perspective presented a limited view of ICT's full environmental scope. Hardware was often considered to be the primary contributor to carbon emissions, while the role of software was largely ignored. As noted by Mocigemba (2006), focusing on hardware exclusively neglects the system-wide interactions that drive the energy demand. Therefore, while significant improvements had been made, overall ICT emissions continued to rise, and it was determined that additional factors were present. This narrow framing of sustainability as a sole hardware problem delayed critical investment in the lifecycle impacts of software systems. This oversight has had profound consequences, particularly as software has become increasingly complex and widespread across multiple industries (Penzenstadler, 2013).

2.2 The Growing Environmental Burden of Software

As ICT systems evolved, software's contribution to environmental deterioration became increasingly evident. Modern software systems, whether they are intelligence algorithms or large-scale cloud infrastructure, require equally substantial computation resources throughout their lifecycle (Vartziotis et al., 2024). According to Vartziotis et al. (2024), the energy required to train a single large-scale model can be equivalent to the annual electricity consumption for 100 homes, highlighting the immense carbon footprint associated with modern software development.

This environmental drain is further exacerbated by the inclusion of data centre energy consumption. According to Naumann et al. (2011), global energy consumption from data centres increased from 58 billion kilowatt-hours (kWh) in 2000 to 123 billion kWh in 2005. This doubling over five years reflects not only the increasing complexities and resource intensities of software but also the hardware demands. Other studies have shown this unsustainable trend, noting that software-driven services outpace gains made in hardware energy efficiency (Mocigemba, 2006; Eeckhout, 2025). Software-induced hardware obsolescence is another variable added to the problem. As newer software versions demand greater processing power and memory, older hardware becomes inadequate despite remaining fully functional (Hilty et al., 2006).

As noted by Eeckhout (2025), the increasing complexity and ubiquity of software systems imply that their environmental footprint cannot be dismissed as minimal. Instead, software must be recognised as a first-class actor in the sustainability conversation, both as a direct source of environmental impact that requires mitigation and as a potential enabler of greener practices.

2.3 Energy Consumption Trends in Modern ICT

The global need for computational power has expanded rapidly over the last two decades, driven mainly by growth in cloud computing, big data analytics, and artificial intelligence. Despite improvements made in processor efficiency or server design, the aggregate energy demand of digital services has continued to grow (Mocigemba, 2006; Eeckhout, 2025). Data centres now consume an estimated 1-1.3% of total global electricity, which amounts to 240-340 terawatt-hours annually

(Vartziotis et al., 2024). Artificial intelligence (AI) workloads represent one of the most energy-driven drivers of this growth. Recent analysis by Eeckhout (2025) identifies that overall carbon footprints continue to grow without substantial intervention. As shown in Figure 1, using the compound annual growth rate (CAGR), more forceful methods of sustainability may be needed to offset increasing usage patterns.

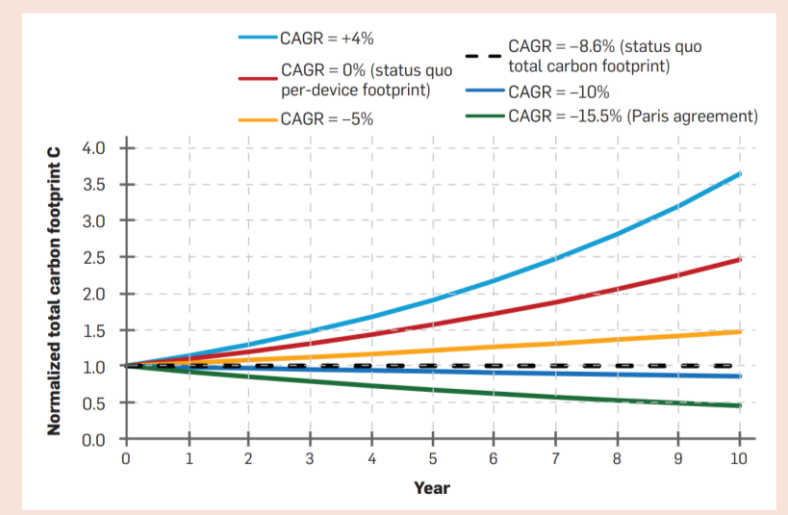


Figure 1 - Total carbon footprint normalized over time for different per-device scaling scenarios (adapted from Eeckhout, 2025)

Training a single large natural language processing model such as OpenAI's GPT-3 reportedly generated over 532 tonnes of CO_2 emissions (Vartziotis et al. 2024; Ceurstemont, 2024). These lifecycle emissions, which include both training and testing phases, often rival or exceed the lifetime emissions count of traditional manufactured goods like automobiles (Ceurstemont, 2024). As applications for AI, blockchain, and big data analytics continue to expand in industry, there is a growing sense to rethink how software engineering practices can be altered beyond hardware optimisations (Becker et al., 2015; Naumann et al., 2011).

2.4 System Effects and Rebound Phenomena

While first-order impacts such as direct energy use and emissions from data centres are increasingly well understood, second and third-order systemic impacts are much more difficult to classify and measure. Second-order effects refer to the indirect consequences of using ICT, such as dematerialisation of services. This could be simplified to how virtual meetings reduce travel or the optimisation of logistics through software (Naumann et al., 2011). Third-order effects involve broader social and economic transformations that evolve over time and produce surprising undesirable environmental outcomes (Hilty et al., 2006). Examples of this include changes in consumer behaviour, accelerated economic growth due to increased efficiency, or the inclusion of new technology sectors.

One major systemic phenomenon affecting ICT sustainability is the rebound effect. Originally described in the context of energy economics, the rebound effect occurs when efficiency improvements lower the cost of a service, leading to increased consumption that partially or fully offsets the environmental gains (Hilty et al., 2006). A similar concept is the Jevons' Paradox mentioned by Eeckhout (2025), where the rebound effect is so strong that it fully cancels out or even reverses savings. This is visible in cloud computing because, as storage and processing become cheaper and more efficient, demand for data-intensive applications like streaming, gaming, or AI has increased. Rather than reducing overall energy consumption, innovations often stimulate greater use and increase environmental pressures. Understanding and addressing these systemic effects is imperative for truly sustainable software engineering. Models such as GREENSOFT model (Naumann et al., 2011) or the Karlskrona Manifesto (Becker et al., 2015) highlight the need to predict

rebound dynamics during the design, deployment, and scaling of digital systems.

2.5 Need for a Holistic Sustainability Approach

Based on earlier sections, it is clear that hardware-only and software-only sustainability measures are inadequate and highlight the need for a comprehensive system-wide approach to sustainability in ICT. Addressing isolated parts of the problem will not suffice if broader lifecycle and system impacts are ignored (Becker et al., 2015; Lau et al., 2014). A truly sustainable digital ecosystem requires the consideration of both reduced energy consumption during operation and the embodied energy for hardware.

Models like the GREENSOFT model (Naumann et al., 2011) propose embedding sustainability principles throughout the entire SDLC, from requirements engineering through to disposal and recycling phases. Similarly, the Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) calls for sustainability to be addressed as a foundational design concern alongside commonplace principles like security, usability, and performance. These perspectives argue that sustainability should be a primary quality attribute tied directly to how a system is conceived, built, and maintained.

3. Foundations of Green Software Engineering

3.1 Defining Green and Sustainable Software Engineering

Early ideas of sustainability in software stemmed from the reasoning that while software was nonphysical, it carried environmental impacts through its design, development, and lifecycle interactions with hardware systems. Dick et al. (2010) provided one of the earliest formal definitions of "Green and Sustainable Software", describing it as software that, post-development, deployment, and usage, has a positive effect on sustainable development regardless of the direct or indirect negative impacts it may generate.

In parallel, a separate but complementary concept, "Green and Sustainable Software Engineering", emerged to focus on the process rather than the product. While green software emphasises the characteristics and impacts of the final software product, green software engineering refers to the embedding of sustainability principles throughout the software development lifecycle (Naumann et al., 2011; Becker et al., 2015). This includes practices such as setting sustainability-oriented requirements, designing modular and resource-efficient architectures, optimising programming for energy consumption, and planning for long-term maintainability to reduce system obsolescence. Frameworks like the GREENSOFT Model (Naumann et al., 2011) and principles outlined in the Karlskrona Manifesto stress that sustainability should be regarded as a continuous engineering objective, comparable to primary attributes like security or usability. Understanding the distinction between sustainable software and sustainable software engineering is crucial for advancing green practices in ICT. Without addressing either property of the software product or the processes in its creation, you risk sustainability interventions being flawed or superficial.

3.2 Green by IT versus Green In IT

A focal distinction within the discussion on sustainable software is the difference between "Green by IT" and "Green in IT." This terminology, similar to what is mentioned in Section 3.1, introduced by Dick et al. (2010) and later expanded on by Naumann et al. (2011), clarifies the dual roles that software can play in environmental sustainability. "Green by IT" refers to the development of software systems that enable or support sustainability in other domains. Examples of this would be smart grid optimisation platforms, transportation management systems that minimise fuel use, or agricultural monitoring applications that enhance resource efficiency (Mahmoud & Ahmad, 2013; Albertao et al., 2010).

Conversely, "Green in IT" focuses on making the ICT sector itself more environmentally sustainable by improving the energy and resource efficiency of software systems internally (Dick et al., 2010). This encompasses practices like optimising algorithmic structures to reduce computation loads, designing software architectures with longevity and adaptability in mind, and minimising the indirect environmental impacts of ICT operations. Frameworks like the Karlskrona Manifesto (Becker et al., 2015) stress that "Green in IT" efforts are imperative as the ICT industry's growth trajectory threatens to negate sustainability gains made elsewhere if left unchecked. Understanding this distinction is crucial for properly framing sustainable software proposals.

3.3 Major Frameworks and Models

Several foundational models have been proposed to integrate sustainability considerations across the software development lifecycle. The GREENSOFT Model (Naumann et al., 2011) is among the most commonly used, offering a reference model that embeds sustainability assessments and actions into every stage of development, deployment, usage, and disposal. Complementing this, Shenoy and Eeratta (2011) introduced the Green Software Development Model, as seen in Figure 2, emphasising practical steps to minimise energy consumption and material usage during development. Mahmoud and Ahmad (2013) further proposed the Green Model for Sustainable Software Engineering, which added a gold-orientated requirements engineering phase to ensure sustainability is addressed in earlier

stages before running through the rest of the lifecycle. These models collectively highlight the importance of integrating environmental considerations systematically, not merely optimising final software products.

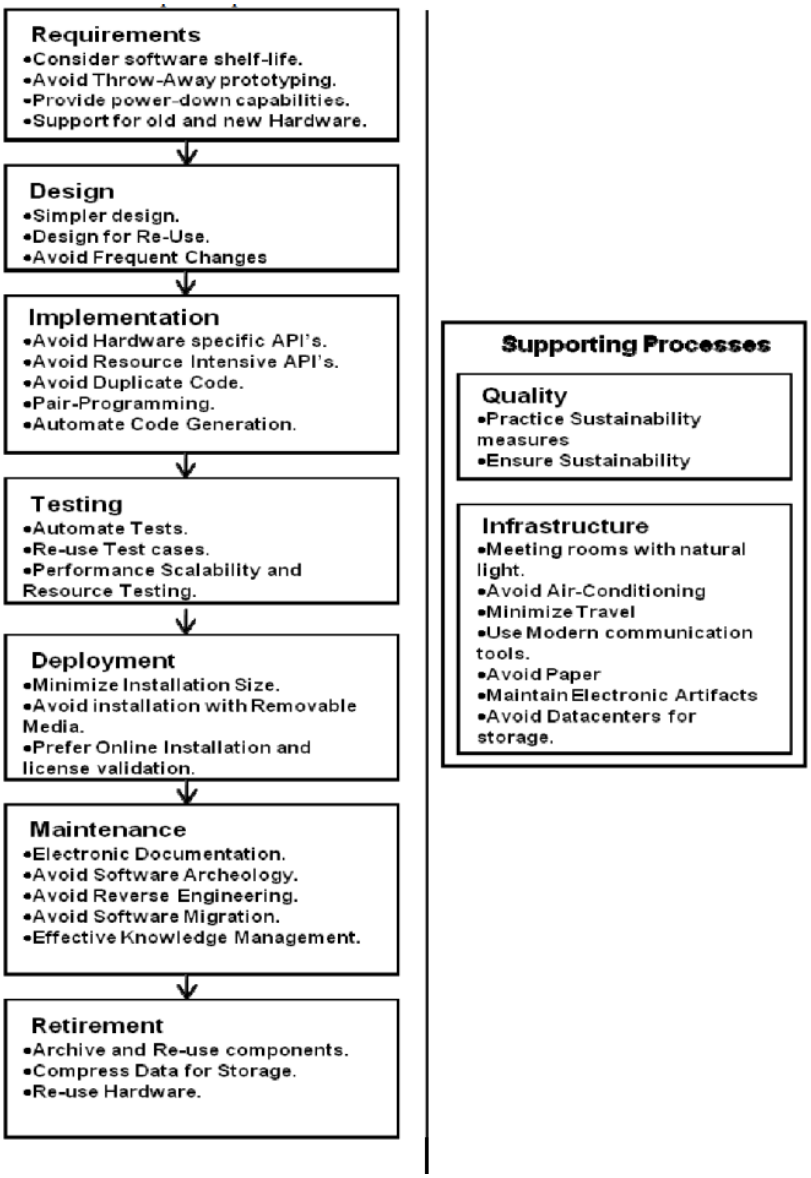


Figure 2 - The Green Software Development Life Cycle Model (Shenoy & Eeratta, 2011)

Beyond process-focused frameworks exist philosophical and evaluative models that expand the perspective of contribution sustainability measures to software. The Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) establishes key principles that underpin much of modern systems thinking in sustainable software engineering. Venters et al. (2014) proposed a framework based on the story "The Blind Men and the Elephant" to assess software sustainability by a method of trial and error, as fragmented perspectives can lead to hindered holistic evaluation. Albertao et al. (2010) contributed a Sustainability Performance Measurement Model, introducing specific metrics to evaluate environmental impacts alongside traditional project management outcomes.

Recent efforts have further defined more diverse models of sustainability. Although earlier contributions like the Green IT Readiness Model by Molla et al. (2009) focused on an organisation's preparedness for eco-sustainability, more recent work has proposed new methods to embed sustainability into process phases. Valmohammadi and Mortaz (2023) proposed a Conceptual Green Process Model, using a mixed methods approach to systematically embed sustainability practices into

process design. These models illustrate the breadth of approaches to sustainable software engineering, ranging from detailed lifecycle frameworks to organisational and cultural readiness assessments. However, a common challenge is still present as a lack of empirical testing in industrial contexts and standardised sustainability metrics across projects remain limited, highlighting the gaps that future research must address.

3.4 Challenges in Defining Metrics

Despite a growing interest in sustainable software engineering among researchers, establishing metrics and standards to evaluate sustainability remains a greater challenge. Early attempts such as the Sustainability Performance Measurement framework proposed by Albertao et al. (2010) suggested a form of integrating environmental considerations into traditional software project evaluations, but limited adoptions have been made. Models like GREENSOFT (Naumann et al., 2011) and the Green Model for Sustainable Software Engineering (Mahmoud & Ahmad, 2013) have included metrics like carbon footprint, energy consumption during execution, and hardware longevity, but a set of universally accepted indicators has not emerged yet. Venters et al. (2014) argued that coherent measurement efforts cannot be introduced without properly considering systemic impacts like energy efficiency.

Several factors contribute to this fragmentation. First, sustainability impacts span multiple dimensions: economic, environmental, and social, which makes adoption inherently more difficult to quantify compared to traditional software quality attributes like performance or security (Becker et al., 2015). Second, contextual factors such as deployment environments, usage patterns, and user behaviours can drastically alter the outcomes of sustainable software systems (Hilty et al., 2006; Kern, 2017). While tools like Greenalytics (Zapico et al., 2010) or the Green Tracker (Amsel & Tomlinson, 2010) offer valuable visualisation methods for raising awareness, there is a lack of standardisation across different software categories. Without a consistent and widely recognised metric, efforts to embed sustainability into software engineering will ultimately fall short.

4. Software Development Life Cycle

The software development life cycle (SDLC) represents a sequential structure for software systems, involving its creation, deployment, and eventual retirement. As software becomes more integrated with energy-intensive infrastructures and device ecosystems, the environmental sustainability of each phase in the SDLC has gained prominence. Industry specialists and researchers have responded to this issue with structured models like the GREENSOFT model (Naumann et al., 2011) as seen in Figure 3, the Green Software Development model (Shenoy & Eeratta, 2011), and the Green Model for Sustainable Software Engineering (Mahmoud & Ahmad., 2013) that embed sustainability features throughout the software life cycle. This section will explore the sustainability implications across the requirements, design, development, testing, deployment, maintenance, and disposal phases and discuss the metrics proposed to assess sustainable software practices.

4.1 Requirements and Design Stages

Embedding sustainability into software begins most effectively at the requirements and design stages. Several models showcase that early interventions offer the greatest leverage for reducing environmental impacts throughout a system's life cycle (Naumann et al., 2011; Mahmoud & Ahmad, 2013). Promoted by Mahmoud and Ahmad (2013), using Goal-Orientated Requirements Engineering (GORE) approaches will involve explicit incorporation of sustainability objectives into functional and non-functional requirements. This involves setting early goals such as minimising energy consumption, supporting backwards compatibility, and reducing update frequency. Similarly, Chitchyan et al. (2016) highlight that requirements engineering that is sustainability-aware must consider trade-offs between immediate system performance and long-term environmental impacts. Similarly, Duboc et al. (2019) state that raising early awareness in requirements engineering is a necessity, as negative factors often arise in early stages of software development, so it is key that sustainability impacts are identified. Without addressing these concerns early on, opportunities for later sustainability-themed optimisation are diminished greatly.

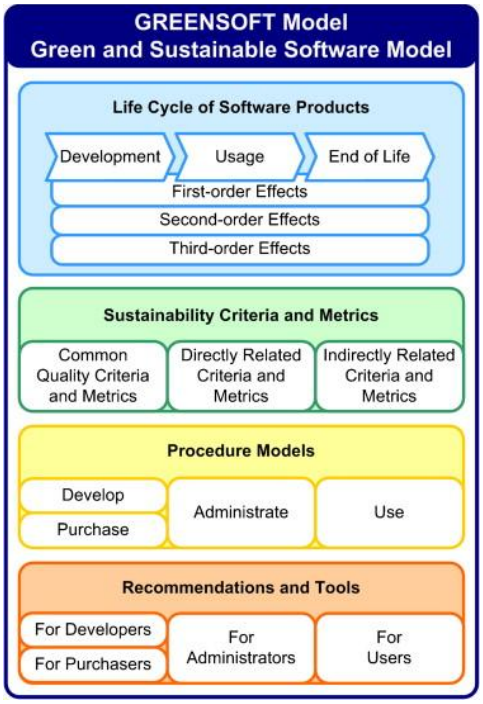


Figure 3 - The GREENSOFT Model, a reference model for "Green and Sustainable Software." (Naumann et al, 2011)

At the design stage, sustainable software engineering models encourage principles such as modularity, simplicity, and reusability. According to the GREENSOFT Model (Naumann et al., 2011) and the Green Software Development Model (Shenoy & Eeratta, 2011), by using modular architectures for

maintenance and selective upgrading, the likelihood of whole system replacement and e-waste generation is reduced. In addition, careful design can optimise user interaction patterns and background processing demands, reducing energy usage (Venters et al., 2014). By embedding sustainability objectives into both requirements and architecture, software projects can proactively align environmental stewardship with core system qualities like usability, scalability, and reliability.

4.2 Development and Testing Stages

During the development phase, sustainability concerns shift from abstract goals to concrete programming practices. Energy-aware coding strategies, such as minimising computational complexity, optimising memory management, and reducing unnecessary database queries, are just a few that are critical to ensuring that applications operate efficiently (Shenoy & Eeratta, 2011; Mahmoud & Ahmad, 2013). The Green Software Development Model (Shenoy & Eeratta, 2011) particularly accentuates adopting coding standards that prioritise low-energy operations, favouring simple algorithms and leaner processing paths. Automated code analysis tools can assist developers in identifying hotspots of energy in code early on. Pair programming and agile code reviews, if supplemented with proper sustainability criteria, can further promote greener development practices by embedding awareness directly into team workflows (Becker et al., 2015).

Testing stages offer further opportunities to assess and improve sustainability before deployment. Tools such as Joulemeter (Kansal et al., 2010) enable developers to measure the energy consumption of individual software components during testing, providing actionable feedback on where optimisations are needed. Mahmoud and Ahmad (2013) argue that integrating sustainability metrics into automated testing, such as energy usage per test case or carbon impact per function, could reveal inefficiencies that traditional performance or validation testing may overlook. Moreover, using cloud-based automation testing environments can reduce unnecessary infrastructure emissions, with the assumption that careful resource provisioning is defined (Zapico et al., 2010). An example of an early tool designed to visualise the environmental impact of web applications is Greenalytics, shown in Figure 4. Although in beta form, it demonstrates the growing perception that software systems carry hidden but measurable carbon costs.

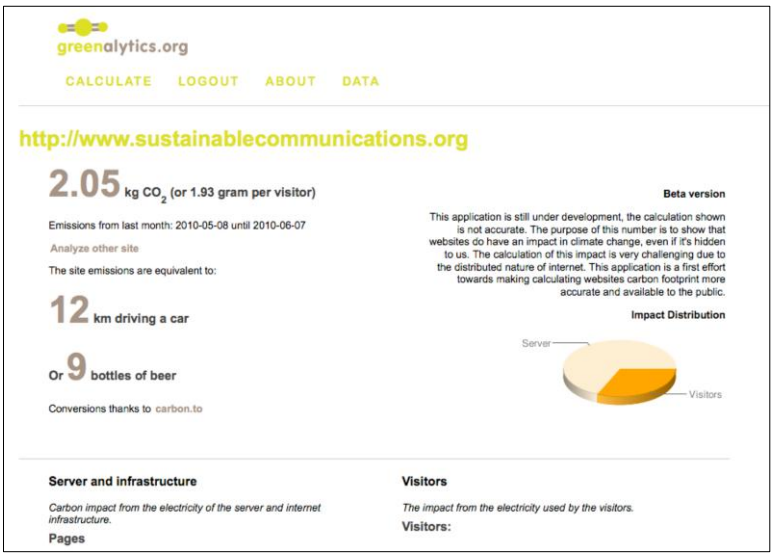


Figure 5 – Greenalytics.org beta from website estimating carbon emissions (adapted from Zapico et al, 2010)

4.3 Deployment, Maintenance and Disposal Stages

The deployment phase introduces sustainability challenges that are often overlooked but are vital for minimising software's environmental footprint. GREENSOFT (Naumann et al., 2011) and the Green

Software Development Model (Shenoy & Eeratta, 2011) both highlight deployment optimisation strategies, such as reducing installation package sizes, using centralised/cloud-based deployments, and lessening redundant data transfers. Large installation files and frequent updates can strain network infrastructure unnecessarily and increase energy consumption, particularly when scaled across millions of devices. Packaging software efficiently, compressing resources, and offering selective modular downloads help to mitigate these impacts (Mahmoud & Ahmad, 2013). Furthermore, considering energy-efficient hosting solutions can significantly lower the emissions associated with software operation post-deployment (Zapico et al., 2010).

Sustainable maintenance and disposal practices aim to extend software longevity and reduce forced obsolescence of hardware. Mahmoud and Ahmad (2013) emphasise the use of sustainable maintenance strategies, including patching, that avoid bloated codebases and retain operational efficiency over time. Disposal considerations are often neglected in software projects as they involve planning for the decommissioning of software systems to minimise the generation of e-waste tied to hardware dependencies. Legacy hardware can be repurposed for less resource-intensive operations, or software can be optimised to extend the lifespan of older devices and align with circular economy practices at the same time.

4.4 Metrics for Sustainable Software

Quantifying metrics within the SDLC requires a multidimensional perspective. Naumann et al. (2011) proposed two metric types across the direct first-order impacts and the indirect second and third-order impacts. The metrics issued in the first order would suggest direct energy usage during execution and carbon footprint during the development and deployment phases of the SDLC, while second- and third-order effects address indirect and systemic effects. Second-order effects would involve how software helps reduce the need for physical processes, like virtual meetings replacing travel. Third-order metrics would involve changes in operational behaviour or observed device usage patterns because of software availability. Naumann et al. (2011) proposed two indicators of green performance: hardware compatibility longevity and the carbon footprint of the lifecycle of a software package.

Other researchers have proposed complementary metrics to capture different aspects of sustainability. Mahmoud and Ahmad (2013) proposed tracking installation package size and update frequency, arguing that smaller, less frequent updates reduce resource consumption and extend hardware viability. Shenoy and Eeratta (2011) pointed out energy consumption per function as a practical metric that is actionable at the development stage. In addition to developing metrics, efforts have been made to introduce sustainable labelling identifiers for software products, aiming to enhance user awareness and transparency about the environmental impact of digital services (Kern et al., 2015). Additional tools like Joulemeter (Kansal et al., 2010), which estimates the energy consumption of virtual machines and individual software processes, and the Green Power Indicator (Zapico et al., 2010), a browser plugin that visualises the energy impact of website usage, provide partial but useful measurements outside traditional SDLC evaluation processes. However, Venters et al. (2014) argue that fragmented and narrowed metrics risk missing systemic impacts unless altered by broader sustainable frameworks.

Overall, while diverse measurement techniques have emerged, the lack of standardised sustainability benchmarks across projects and industries remains a massive barrier. Future research must address the need for a comprehensive, context-sensitive metric that evaluates both operational energy use and lifecycle behaviour and social norms of software sustainability.

5. Software Abandonment and Large-Scale Failures

5.1 Causes of Abandonment

Large-scale software project failures often arise from a combination of technical, managerial, and strategic misalignments. Albertao et al. (2010) and Drummond (1999) both emphasise that runaway complexity, scope creep, inadequate requirements gathering, and changing stakeholder demands are common drivers of project escalation and eventual ruin. Poor risk management, weak stakeholder communication, and unrealistic timelines frequently complicate these issues and lead to the system failing to meet original goals or becoming obsolete before completion. From a sustainability viewpoint, such failures are a result of not representing environmental and economic costs. Abandoned systems consume resources during their development, generate emissions from their infrastructure use, and finally contribute heavily to e-waste from discarded hardware and software components (Venters et al., 2014; Becker et al., 2015).

Several historic cases illustrate these challenges vividly. The Customer Account Data Engine (CADE) project, which ran from 2000 to 2009 in the United States and intended to modernise tax processing, faced extensive delays and system integration issues, leading to eventual abandonment and major resource wastage. Similarly, the CSIO Portal in Canada ran from 1999 to 2006. They struggled with constant changing system requirements, security concerns, and poor architectural planning, leading to significant wasted budget. Both examples highlight sustainability failures that are not only technical but also systemic blunders, emerging when projects fail to understand long-term operational viability and adaptability. Recognising these patterns provides critical context for understanding the failures of other prominent projects such as the FBI's Virtual Case File (VCF), the UK e-Borders system, and the Taurus project, which will be discussed in the following sections.

5.2 FBI Virtual Case File

The FBI VCF project was launched in 2000 with the objective of modernising the agency's outdated paper-based case management system. The system was intended to allow agents and analysts to electronically store, retrieve, and share intelligence data more effectively and efficiently across the organisation (Marchewka, 2014). Over its development, the VCP project underwent many major revisions, expanding beyond its original scope and requiring frequent redesigns. By 2005, after approximately 170 million United States Dollars (USD) had been spent, the project was abandoned before reaching operational status. The abandoned VCF infrastructure was eventually replaced by a separate system, the Sentinel project.

5.3 UK e-Borders

The UK e-Borders program, initially formed in 2003, aimed to create an advanced passenger information system capable of screening individuals before their arrival into the United Kingdom. Managed initially by the UK Home Office and various private contractors, the project was envisioned with ambitions of integrating immigration, customs, and security data across multiple agencies to improve border security and operational efficiency (Alami, 2016a). The scope of e-Borders includes collecting and analysing extensive passenger information from airlines and other travel providers, enabling risk-based assessments in real time.

Throughout its development, the program underwent multiple internal structural changes, including contract shifts, policy changes, and integration attempts with legacy systems. Despite significant investments, reportedly exceeding 830 million British pounds (GBP), the e-Borders system was never fully deployed as envisioned. By 2015, much of the project has been dismantled or absorbed into successor projects.

5.4 Taurus

The Transfer and Automated Registration of Uncertified Stock (Taurus) project, launched by the

London Stock Exchange in the late 1980s, was aimed at automating the settlement and registration of share transactions, replacing the paper-based system that was existing at the time. It was designed to create an electronic platform that would streamline trading operations, reduce administrative costs, and enhance market efficiency (Drummond, 1999).

Initially budgeted at six million GBP, Taurus quickly expanded in scope and complexity, as demands from various financial institutions requested additional features and customisations. The project experienced numerous delays, continual revisitations, and technical integration difficulties. After several years of development and an estimated 400 million GBP in accumulated costs across stakeholders, the Taurus project was formally abandoned in 1993 without delivering a working system.

5.5 Reason for project failure

Across the FBI VCF, UK e-Borders, and Taurus projects, several reoccurring causes of failure are evident. One major issue was uncontrolled scope expansion, where initial project goals were continually revisited without corresponding amendments in planning or design (Marchewka, 2014; Alami, 2016a; Drummond, 1999). In all three cases, shifting requirements overwhelmed the original system designs, leading to great complexity and technical instability. Poor stakeholder management further compounded these problems, with communication gaps between developers, end users, and sponsoring agencies meaning that systems evolved without fully addressing operational needs. Technical overstretching also played a significant role as projects attempted to integrate outdated legacy systems or deploy unproven techniques without sufficient prototyping. Observations that can be noticed from these three projects align with broader patterns identified in ICT project failure literature, which highlight insufficient risk management, unrealistic timelines, and lack of adaptability to key contributors (Alami, 2016b).

A sustainability-oriented approach could have significantly mitigated these failures. Early modular design, with emphasis on flexibility and incremental deliverables, would have allowed the projects to adapt more gracefully to the changing requirements without requiring complete system overhauls. GORE could have helped clarify essential versus non-essential features, keeping the projects aligned with core needs (Mahmoud & Ahmad, 2013). Sustainability metrics, such as monitoring infrastructure energy efficiency, might have triggered earlier interventions to reduce unnecessary resource consumption (Albertao et al., 2010; Zapico et al., 2010). Embedding sustainability principles such as adaptability, minimal viable architecture, and lifecycle impact awareness into the software engineering process would have offered a path forward that would have prevented the failure of the projects. These failures carried a substantial environmental cost, where resources used in development, such as electricity, hardware infrastructure, and computation, were expended and ultimately wasted. Understanding these fatal flaws of hidden environmental impacts further underscores the need to implement sustainability practices into the earliest stages of software system design.

6. Green Software Adoption in Industry Challenges

Despite growing awareness of the environmental footprint of software systems, integration of green software practices into mainstream industry remains limited. While academic models such as GREENSOFT (Neumann et al. 2011) and the Green Software Development Model (Shenoy & Eeratta 2011) offer detailed pathways for embedding sustainability into the software lifecycle, practical methods of adoption for industry are obstructed by a broad set of real-world constraints. These include economic pressures, lack of awareness, insufficient tooling, and increased misconceptions about trade-offs between sustainability, performance, and profitability. This section will discuss the key barriers inhibiting adoption of sustainable software engineering practices in industry.

6.1 Economic and Organisational Constraints

One of the most significant impediments to the adoption of green software engineering is the lack of financial incentives. Organisational profits are often tied to short-term financial performance rather than long-term sustainability benefits, particularly when the cost savings from energy-efficient software are not immediately visible (Mahmoud & Ahmad, 2013). Benefits, such as reduced energy consumption, extended hardware lifespan, and future compliance with environmental regulations, are difficult to quantify within conventional return on investment models. Furthermore, sustainability is often treated as a secondary concern rather than a core project initiative, leading to minimal resource allocation during planning stages (Valmohammadi & Mortaz, 2023). Software development structures that prioritise speed through agile sprints or continuous integration/development further discourage sustainability improvements. Additional factors, such as the need to rethink infrastructure, retrain personnel, or potentially delay release cycles, also deter organisations from integrating sustainable practices.

6.2 Lack of Awareness and Tooling

Another major challenge lies in the low level of awareness among software professionals regarding the environmental impacts of software decisions. Many developers, project managers, and even company executives fail to recognise and acknowledge sustainability as a relevant design concern (Kern, 2017). Broader survey results further illustrate the gap between awareness and practical knowledge in the ICT sector. As shown in Table 1, while knowledge of green computing exists among various groups, consistent sustainable practices remain lacking (Kern, 2017). As a result, few organisations will include green metrics in performance evaluations or in software quality benchmarks. As a result, sustainability metrics are rarely implemented into software quality benchmarks or performance evaluations. Tools like Joulemeter (Kansal et al., 2010), Greenalytics (Zapico et al., 2010), and the Green Power Indicator (Zapico et al., 2010) have been developed to measure energy usage, but they are not widely integrated into mainstream development environments. Mahmoud and Ahmad (2013) argue that green software tooling is typically fragmented and often seen as a supplementary aspect rather than essential. This lack of sustainability indicators and benchmarks increases the difficulty of the problem. Although green performance indicators have been proposed in academic literature (Neumann et al., 2011), they are not widely enforced by industry professionals, which leaves organisations without proper references for measurements.

Target group	Objective and focus	Findings	Source
Programmers	Do programmers know about energy consumption of *software* and how to reduce it although such issues are not addressed in education or trainings?	Programmers had limited knowledge of energy efficiency, lacked knowledge of the best practices to reduce energy consumption and were unsure about how software consumes energy	Pang et al. (2016)
Practitioners	Relationship between research community and practitioners: same interests? Asking for the practitioners' perspective to get a motivation and guide for green *software* engineering research	Practitioners care about energy but are not as effective as they could be at creating efficient applications	Manotas et al. (2016)
Students in Mauritius	Analyze the awareness of green computing (focus: *hardware*) among students	Students have a moderate knowledge about green computing but their everyday green computing practices are not satisfactory	Dookhitram et al. (2012)
Students in Malaysia	*Hardware* focus: usage and selling of computers and related resources	Students have an average level awareness about green computing and their everyday practices are not satisfactory	Selyamani and Ahmad (2015)
Buyer and supplier	Analyze people's awareness of *software* engineering issues	People are aware of the importance of energy efficient software, but lack information and resources to engage this topic	Ferreira (2011)
ICT Managers	How do ICT managers implement energy efficient methods (focusing on *hardware* issues) in their organizations?	Just over a third of organisations in Europe do not implement green IT practices	Kogelman (2011)

Table 1—Overview of studies addressing the awareness of green computing issues (adapted from Kern, 2017).

6.3 Cultural and Technical Misconceptions

A persistent cultural barrier is the perception that green software engineering compromises performance, innovation, or usability. For instance, optimising code for energy efficiency is often assumed to reduce computational speed or limit design richness (Becker et al., 2015). There is also significant resistance to change in many engineering environments, especially in agile or lean teams where flexibility and rapid iteration are highly valued. Introducing sustainability as a design concern is sometimes viewed as an impediment to these workflows. Moreover, even in organisations that are aware of environmental issues, sustainability tends to be ranked lower than traditional project priorities such as reliability, scalability, or maintainability (Naumann et al., 2011). These misconceptions perpetuate a cycle where sustainability is seen as optional rather than a fundamental quality attribute.

6.4 Trade-offs and Complexity in Real-World Projects

Even when sustainability is acknowledged as being vital, real-world initiatives still require challenging trade-offs for practical implementation. Project managers and developers frequently find it difficult to strike a balance between conflicting goals: sustainability goals must coexist with client deadlines, financial limitations, user experience requirements, and system performance standards (Venters et al., 2014). Often, especially in fiercely competitive business settings, short-term functional needs take precedence over long-term sustainability objectives. According to Kern (2017), this tension is made worse by the lack of sustainability evaluation tools, which prevent teams from measuring and defending green design choices objectively inside routine processes. In addition to better tools, project management techniques must change to incorporate sustainability standards from the very beginning of planning to overcome these real-world obstacles and make sure they are not compromised by operational demands.

7. Summary

This literature review examined how sustainability and software engineering are changing together, emphasising how crucial it is to incorporate environmental factors into every stage of the software development life cycle. At first, ICT sustainability initiatives were mostly concerned with hardware efficiency, but as the environmental impact of software systems increased, focus was switched to sustainable and green software engineering techniques. From requirements collection to deployment and disposal, many methods of integrating sustainability are illustrated by foundational models, including GREENSOFT, the Green Software Development Model, and the Green Model for Sustainable Software Engineering, to name a few. The FBI Virtual Case File, the UK e-Borders system, and the Taurus project are just a few instances of large-scale software failures that illustrate the significant resource and environmental consequences linked to inadequate early lifecycle planning and technical misalignment. Furthermore, economic pressures, ignorance, fragmented tooling, and cultural misunderstandings regarding trade-offs between sustainability and essential software features continue to hinder industry adoption of sustainable practices.

There are still large gaps in both research and practice, despite an increase in scholarly contributions. Software sustainability metrics are still dispersed and lack industry-wide standardisation, which makes it difficult to compare and evaluate projects in a meaningful way. Furthermore, current frameworks frequently fail to adequately handle systemic impacts like the rebound effect or the more extreme Jevons' paradox, which occurs when enhanced efficiency results in higher overall resource use. Adoption is made more difficult by practical obstacles in real-world initiatives, such as strict commercial deadlines and a lack of integration of sustainability indicators into routine procedures. Green software engineering runs the risk of staying a novel concept rather than becoming the standard in the industry if sustainability is not treated as a fundamental quality factor rather than an optional condition.

The goal of future work should be to create usable, standardised sustainability criteria that are easily included in regular software engineering procedures. Building solid, scalable solutions will need a transdisciplinary approach, where environmental, social, economic, and software engineering can coexist. Priority should be given to educational programmes aimed at increasing awareness among executives, project managers, and developers. Putting sustainability at the core of software engineering is both an opportunity and a responsibility to make sure that technical advancements promote rather than impede global environmental goals as digitalisation continues to transform society.

References

Alami, A. (2016a). The UK e-Borders Project Failure. In *PM World Journal The UK e-Borders Project Failure*. https://pmworldlibrary.net/wp-content/uploads/2016/03/pmwj44-Mar2016-Alami-uk-eborders-project-failure-featured-paper.pdf

Alami, A. (2016b). Why Do Information Technology Projects Fail? *Procedia Computer Science*, *100*, 62–71. https://doi.org/10.1016/j.procs.2016.09.124

Albertao, F., Xiao, J., Tian, C., Lu, Y., Zhang, K. Q., & Liu, C. (2010). Measuring the Sustainability Performance of Software Projects. *2010 IEEE 7th International Conference on E-Business Engineering*, 369–373. https://doi.org/10.1109/icebe.2010.26

Amsel, N., & Tomlinson, B. (2010). Green tracker. *10 Extended Abstracts on Human Factors in Computing Systems*. https://doi.org/10.1145/1753846.1753981

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., & Venters, C. C. (2015). Sustainability Design and Software: The Karlskrona Manifesto. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. https://doi.org/10.1109/icse.2015.179

Bekaroo, G., Bokhoree, C., & Pattinson, C. (2013). Towards Green IT Organisations: A Framework for Energy Consumption and Reduction. *ResearchGate*, *8*(3). https://doi.org/10.18848/1832-3669/CGP/v08i03/56290

Ceurstemont, S. (2024). Controlling AI's Growing Energy Needs. *Communications of the ACM*, *68*(3), 15–17. https://doi.org/10.1145/3703788

Chitchyan, R., Becker, C., Betz, S., Duboc, L., Penzenstadler, B., Seyff, N., & Venters, C. C. (2016). Sustainability design in requirements engineering. *Proceedings of the 38th International Conference on Software Engineering Companion - ICSE '16*. https://doi.org/10.1145/2889160.2889217

Deng, Q., & Ji, S. (2015). Organizational Green IT Adoption: Concept and Evidence. *Sustainability*, *7*(12), 16737–16755. https://doi.org/10.3390/su71215843

Dick, M., & Naumann, S. (2010). *Enhancing Software Engineering Processes towards Sustainable Software Product Design* (pp. 706–715). EnviroInfo. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=98d04de0318b252f273e3cf36cc385253542b924

Dick, M., Naumann, S., & Kuhn, N. (2010). *A Model and Selected Instances of Green and Sustainable Software*. 248–259. https://doi.org/10.1007/978-3-642-15479-9_24

Drummond, H. (1999). Are we any closer to the end? Escalation and the case of Taurus. *International Journal of Project Management*, *17*(1), 11–16. https://doi.org/10.1016/s0263-7863(97)00074-4

Duboc, L., Betz, S., Penzenstadler, B., Kocak, S. A., Chitchyan, R., Leifler, O., Porras, J., Seyff, N., & Venters, C. C. (2019, September 1). *Do we Really Know What we are Building? Raising Awareness of Potential Sustainability Effects of Software Systems in Requirements Engineering*. IEEE Xplore. https://doi.org/10.1109/RE.2019.00013

Eeckhout, L. (2025). The Sustainability Gap for Computing: Quo Vadis? *Communications of the ACM*, *68*(3), 70–79. https://doi.org/10.1145/3699595

Hilty, L. M., Köhler, A., von Schéele, F., Zah, R., & Ruddy, T. F. (2006). Rebound effects of progress in information technology. *Poiesis & Praxis*, *4*(1), 19–38. https://doi.org/10.1007/s10202-005-0011-2

Kansal, A., Zhao, F., Liu, J., Kothari, N., & Bhattacharya, A. (2010). Virtual machine power metering and provisioning. *Symposium on Cloud Computing*. https://doi.org/10.1145/1807128.1807136

Kern, E. (2017). Green Computing, Green Software, and Its Characteristics: Awareness, Rating, Challenges. *Progress in IS*, 263–273. https://doi.org/10.1007/978-3-319-65687-8_23

Kern, E., Dick, M., Naumann, S., & Filler, A. (2015). Labelling sustainable software products and websites: Ideas, Approaches, and Challenges. *Advances in Computer Science Research*. https://doi.org/10.2991/ict4s-env-15.2015.10

Lau, L. M. S., Griffiths, M., Holmes, V., Ward, R., Jay, C., Dibsdale, C., Xu, J., Austin, J., & Venters, C. C. (2014). Software Sustainability: The Modern Tower of Babel. *RE4SuSy: Third International Workshop on Requirements Engineering for Sustainable Systems*, *1216*. https://www.researchgate.net/publication/303617265_Software_Sustainability_The_Modern_Tower_of_Babel

Mahmoud, S. S., & Ahmad, I. (2013). A green model for sustainable software engineering. *International Journal of Software Engineering and Its Applications*, *7*(4), 55–74. https://www.researchgate.net/publication/285762532_A_green_model_for_sustainable_software_engineering

Marchewka, J. T. (2014). The FBI Virtual Case File: A Case Study. *Communications of the IIMA*, *10*(2). https://doi.org/10.58729/1941-6687.1132

Mayr, H. C., Pinzger, M., & Kern, E. (2016). *The Development of an Eco-Label for Software Products - a Transdisciplinary Process?* (p. 1285). Gesellschaft für Informatik. https://cs.emis.de/LNI/Proceedings/Proceedings259/1285.pdf

Mocigemba, D. (2006). Sustainable Computing. *Poiesis & Praxis*, *4*(3), 163–184. https://doi.org/10.1007/s10202-005-0018-8

Molla, A., Cooper, V. A., & Pittayachawan, S. (2009). IT and eco-sustainability: Developing and validating a green IT readiness model. *ResearchGate*. https://doi.org/10.13140/2.1.5004.1124

Naumann, S., Dick, M., Kern, E., & Johann, T. (2011). The GREENSOFT Model: A reference model for green and sustainable software and its engineering. *Sustainable Computing: Informatics and Systems*, *1*(4), 294–304. https://doi.org/10.1016/j.suscom.2011.06.004

Penzenstadler, B. (2013). *Towards a definition of sustainability* in *and* for *software engineering*. https://doi.org/10.1145/2480362.2480585

Shenoy, Sanath. S., & Eeratta, R. (2011). Green software development model: An approach towards sustainable software development. *2011 Annual IEEE India Conference*. https://doi.org/10.1109/indcon.2011.6139638

Valmohammadi, C., & Mortaz Hejri, F. (2023). Designing a conceptual green process model in software development: A mixed method approach. *International Journal of Information Management Data Insights*, *3*(2), 100204. https://doi.org/10.1016/j.jjimei.2023.100204

Vartziotis, T., Dellatolas, I., Dasoulas, G., Schmidt, M., Schneider, F., Hoffmann, T., Kotsopoulos, S., & Keckeisen, M. (2024). Learn to Code Sustainably: An Empirical Study on Green Code Generation. *Proceedings of the 1st International Workshop on Large Language Models for Code*, 30–37. https://doi.org/10.1145/3643795.3648394

Venters, C. C., Lau, L., Griffiths, M. K., Holmes, V., Ward, R. R., Jay, C., Dibsdale, C. E., & Xu, J. (2014). The Blind Men and the Elephant: Towards an Empirical Evaluation Framework for Software Sustainability. *Journal of Open Research Software*, *2*(1). https://doi.org/10.5334/jors.ao

Zapico, J. L., Turpeinen, M., & Brandt, N. (2010). *Greenalytics: a tool for mash-up life cycle assessment of websites.* ResearchGate. https://www.researchgate.net/publication/229037499_Greenalytics_a_tool_for_mash-up_life_cycle_assessment_of_websites